Planejamento Lógico TaskTracker - Lucas Cerqueira Lima

Bootcamp II | Entrega Inicial (Fase 1)

Modalidade: EAD — Disciplina Virtual (DV) • Tipo: Individual • Valor: 10,0 pontos

1. Apresentação e Entendimento do Problema

1.1 Identificação

Campo	Informação
Nome completo	Lucas Cerqueira Lima
Curso	Ciência de Dados e Machine Learning
Polo / Turma	2º Semestre, Turma A
E-mail institucional	lucas.limac@sempreceub.com
Disciplina	Bootcamp II
Atividade	Desafio — Entrega Inicial (Fase 1: Planejamento Lógico)

1.2 O Problema a Ser Resolvido

No dia a dia, a maioria das pessoas organiza suas tarefas de forma dispersa: um bilhete no papel, uma mensagem enviada para si mesmo no celular, um lembrete que some no meio de outras notificações e, com frequência, apenas a memória. Esse modelo funciona enquanto o volume é pequeno, mas quebra assim que a rotina cresce.

Os sintomas desse problema são concretos e facilmente observáveis:

- **Esquecimento.** Tarefas importantes se perdem porque não existe um único lugar confiável onde todas estejam registradas.
- **Ausência de prioridade.** Sem uma classificação explícita, tudo parece igualmente urgente e a pessoa acaba resolvendo o que é fácil em vez do que é importante ou prioridade atual.
- **Falta de visibilidade do progresso.** Não há como responder rapidamente a perguntas simples como "o que ainda falta?" ou "quanto eu já concluí?".
- **Retrabalho e informação duplicada.** A mesma tarefa aparece anotada em dois ou três lugares, com versões diferentes e nenhuma delas confiável.
- **Custo mental.** Manter tudo "na cabeça" consome atenção que deveria estar sendo usada para executar as tarefas.

Em um contexto de equipe, o problema se agrava: sem um registro comum e padronizado, ninguém sabe o que o outro está fazendo, tarefas são executadas em duplicidade e prazos passam despercebidos.

O ponto central é que **não se trata de falta de esforço, e sim de falta de sistema**. Falta uma estrutura simples que armazene as tarefas, imponha um mínimo de padronização e devolva a informação organizada.

1.3 A Solução Proposta (TaskTracker)

O **TaskTracker** é um sistema de gerenciamento de tarefas que centraliza o registro, a consulta e o acompanhamento das atividades do usuário em um único lugar, com regras claras de preenchimento.

Nesta Fase 1 o sistema é planejado como uma aplicação de linha de comando (terminal), operada por um menu de opções, escolha que mantém o foco na lógica do problema, e não na interface. O que o sistema fará por quem o usa:

1. **Cadastrar tarefas** com título, descrição, prioridade (Alta, Média ou Baixa) e data limite, validando os dados no momento da digitação.
2. **Listar as tarefas** de forma organizada, ordenadas por prioridade, exibindo claramente o que está pendente e o que já foi concluído.
3. **Marcar tarefas como concluídas**, preservando o histórico em vez de simplesmente apagar o registro.
4. **Filtrar a visualização** por situação (pendentes / concluídas) e por prioridade.
5. **Apresentar um resumo** com o total de tarefas, quantas estão pendentes e quantas foram concluídas.

Em uma frase: *o TaskTracker transforma uma lista mental e desorganizada de compromissos em um registro estruturado, priorizado e consultável.*

Benefício direto: o usuário deixa de depender da memória, passa a saber o que fazer primeiro e consegue visualizar seu próprio progresso.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

São as informações que o usuário digita no sistema.

#	Entrada	Tipo	Obrigatório	Descrição / Exemplo
E1	Opção do menu	Número inteiro	Sim	Ação desejada. Ex.: 1 (cadastrar), 2 (listar), 3 (concluir), 4 (resumo), 0 (sair)

#	Entrada	Tipo	Obrigatório	Descrição / Exemplo
E2	Título da tarefa	Texto	Sim	Nome curto que identifica a tarefa. Ex.: Estudar comandos Git
E3	Descrição da tarefa	Texto	Não	Detalhamento opcional. Ex.: Revisar add, commit, push e pull
E4	Prioridade	Texto (lista fechada)	Sim	Somente Alta, Média ou Baixa
E5	Data limite	Data (DD/MM/AAAA)	Sim	Prazo final da tarefa. Ex.: 13/09/2026
E6	Identificador da tarefa	Número inteiro	Sim (ao concluir)	Código da tarefa que será marcada como concluída. Ex.: 3
E7	Filtro de listagem	Texto / número	Não	Critério de exibição. Ex.: pendentes, concluídas, todas
E8	Confirmação	Texto (S / N)	Sim (ações críticas)	Confirma operações como concluir uma tarefa ou sair do sistema

2.2 Saídas de Dados

São as informações que o sistema exibe na tela.

#	Saída	Descrição / Exemplo
S1	Menu principal	Lista numerada das opções disponíveis
S2	Mensagem de sucesso no cadastro	Tarefa "Estudar comandos Git" cadastrada com sucesso! (ID 3)
S3	Mensagem de erro / validação	Erro: o título da tarefa não pode ficar vazio.
S4	Lista de tarefas organizada	Tabela com ID, título, prioridade, data limite e situação, ordenada por prioridade
S5	Aviso de lista vazia	Nenhuma tarefa cadastrada até o momento.
S6	Confirmação de conclusão	Tarefa 3 marcada como CONCLUÍDA.
S7	Resumo estatístico	Total: 8 tarefas | Pendentes: 5 | Concluídas: 3
S8	Mensagem de encerramento	Encerrando o TaskTracker. Até logo!

2.3 Regras do Sistema

#	Regra	Justificativa
R1	O sistema **não pode aceitar tarefas sem título**. Se o campo vier vazio ou apenas com espaços, exibe erro e solicita novamente.	Uma tarefa sem título é um registro inútil: não identifica nada.

#	Regra	Justificativa
R2	A prioridade deve ser **obrigatoriamente** Alta, Média ou Baixa. Qualquer outro valor é rejeitado e o sistema pede a informação de novo.	Garante que a ordenação por prioridade sempre funcione, sem valores imprevistos.
R3	Cada tarefa recebe um **identificador (ID) único, numérico e sequencial**, gerado automaticamente pelo sistema e nunca reaproveitado.	Permite referenciar a tarefa de forma inequívoca ao concluí-la ou editá-la.
R4	Toda tarefa nasce com a situação **"Pendente"**; a conclusão só ocorre por ação explícita do usuário.	Evita que uma tarefa seja considerada resolvida por engano.
R5	O sistema **não exclui** tarefas concluídas: elas mudam de situação para "Concluída" e continuam registradas.	Preserva o histórico e permite medir o progresso.
R6	A **data limite** deve seguir o formato DD/MM/AAAA e ser uma data válida do calendário.	Impede registros como 31/02/2026, que quebrariam qualquer ordenação por prazo.
R7	Não é permitido concluir uma tarefa **inexistente** ou **já concluída**; nesses casos o sistema exibe mensagem de aviso.	Protege contra operações sem efeito e informa o usuário do que realmente aconteceu.
R8	A listagem é sempre ordenada por **prioridade (Alta → Média → Baixa)** e, em caso de empate, pela **data limite mais próxima**.	Entrega ao usuário a informação já na ordem em que ela deve ser executada.

3. Receita Lógica do Sistema (Algoritmo em Português)

O algoritmo abaixo descreve, em linguagem simples e sequencial, como o sistema funcionará.

3.0 Fluxo Principal (Menu)

```
INÍCIO
  1. Criar uma lista vazia para armazenar as tarefas.
  2. Criar um contador de identificadores começando em 1.
  3. REPETIR enquanto o usuário não escolher sair:
       3.1. Exibir o menu principal com as opções:
            [1] Cadastrar tarefa
            [2] Listar tarefas
            [3] Concluir tarefa
            [4] Ver resumo
            [0] Sair
       3.2. Ler a opção digitada pelo usuário.
       3.3. SE a opção for 1  -> executar o "Passo a Passo do Cadastro".
            SE a opção for 2  -> executar o "Passo a Passo da Visualização".
            SE a opção for 3  -> executar o "Passo a Passo da Conclusão".
            SE a opção for 4  -> executar o "Passo a Passo do Resumo".
            SE a opção for 0  -> exibir mensagem de despedida e encerrar.
            SENÃO             -> exibir "Opção inválida" e voltar ao menu.
FIM
```

3.1 Passo a Passo do Cadastro

```
INÍCIO DO CADASTRO
  1. Pedir ao usuário o TÍTULO da tarefa.
  2. Verificar se o título não está vazio.
       - Se estiver vazio, exibir "Erro: o título não pode ficar vazio"
```

```
        e voltar ao passo 1.
  3. Pedir a DESCRIÇÃO da tarefa (campo opcional; pode ficar em branco).
  4. Pedir a PRIORIDADE da tarefa (Alta, Média ou Baixa).
  5. Verificar se a prioridade digitada é uma das três aceitas.
       - Se não for, exibir "Erro: prioridade inválida. Use Alta,
         Média ou Baixa" e voltar ao passo 4.
  6. Pedir a DATA LIMITE no formato DD/MM/AAAA.
  7. Verificar se a data é válida.
       - Se não for, exibir "Erro: data inválida" e voltar ao passo 6.
  8. Gerar um IDENTIFICADOR único para a tarefa usando o contador atual.
  9. Definir a SITUAÇÃO da tarefa como "Pendente".
 10. Salvar a tarefa (id, título, descrição, prioridade, data limite,
     situação) na lista de tarefas.
 11. Somar 1 ao contador de identificadores.
 12. Exibir a mensagem de sucesso com o título e o identificador.
 13. Voltar ao menu principal.
FIM DO CADASTRO
```

3.2 Passo a Passo da Visualização

```
INÍCIO DA VISUALIZAÇÃO
  1. Acessar a lista de tarefas.
  2. Verificar se existe alguma tarefa cadastrada.
  3. Se a lista estiver vazia:
       - Avisar o usuário com "Nenhuma tarefa cadastrada até o momento"
         e voltar ao menu principal.
  4. Se houver tarefas:
       4.1. Perguntar qual filtro o usuário deseja
            (todas / apenas pendentes / apenas concluídas).
       4.2. Separar da lista somente as tarefas que atendem ao filtro.
       4.3. Se nenhuma tarefa atender ao filtro, avisar o usuário
            e voltar ao menu.
       4.4. Ordenar as tarefas selecionadas por prioridade
            (Alta primeiro, depois Média, depois Baixa) e, em caso de
            empate, pela data limite mais próxima.
       4.5. Exibir o cabeçalho da listagem
            (ID | Título | Prioridade | Data limite | Situação).
       4.6. Mostrar as tarefas uma por uma na tela, seguindo a ordem.
  5. Voltar ao menu principal.
FIM DA VISUALIZAÇÃO
```

3.3 Passo a Passo da Conclusão de Tarefa

```
INÍCIO DA CONCLUSÃO
  1. Exibir a lista de tarefas PENDENTES para o usuário escolher.
  2. Se não houver tarefas pendentes, avisar o usuário e voltar ao menu.
  3. Pedir ao usuário o IDENTIFICADOR da tarefa que deseja concluir.
  4. Procurar essa tarefa na lista.
  5. Se a tarefa não existir, exibir "Tarefa não encontrada"
     e voltar ao passo 3.
  6. Se a tarefa já estiver concluída, avisar o usuário e voltar ao menu.
  7. Pedir confirmação ao usuário (S/N).
  8. Se a resposta for "S", alterar a situação da tarefa para "Concluída"
     e exibir a mensagem de confirmação.
  9. Se a resposta for "N", cancelar a operação sem alterar nada.
 10. Voltar ao menu principal.
FIM DA CONCLUSÃO
```

3.4 Passo a Passo do Resumo

```
INÍCIO DO RESUMO
  1. Contar quantas tarefas existem no total.
  2. Contar quantas tarefas estão com a situação "Pendente".
  3. Contar quantas tarefas estão com a situação "Concluída".
  4. Calcular o percentual de conclusão
```

```
    (concluídas dividido pelo total, multiplicado por 100).
  5. Exibir na tela o total, as pendentes, as concluídas e o percentual.
  6. Voltar ao menu principal.
FIM DO RESUMO
```

4. Mapeamento Arquitetural para Git e GitHub

4.1 Desenho da Árvore do Repositório

Estrutura mínima que será enviada ao GitHub no início da Fase 2:

```
tasktracker-projeto/
|-- README.md                     # Apresentação do projeto no GitHub
|-- .gitignore                    # Ignora arquivos temporários
|-- docs/
|   `-- planejamento_logico.pdf   # Este planejamento da Fase 1
`-- src/
    `-- main.py                   # Arquivo principal do programa Python
```

Estrutura ampliada prevista para o final da Fase 2 (à medida que o código crescer, a organização acompanha):

```
tasktracker-projeto/
|-- README.md                     # Objetivo, instalação e execução
|-- .gitignore                    # Ignora __pycache__/, venv/, *.log
|-- requirements.txt              # Bibliotecas necessárias ao projeto
|-- docs/
|   |-- planejamento_logico.pdf   # Documento da Fase 1
|   `-- algoritmo.md              # Receita lógica em formato editável
|-- src/
|   |-- main.py                   # Ponto de entrada: menu e chamadas
|   |-- tarefas.py                # Cadastrar, listar, concluir, resumir
|   |-- validacoes.py             # Verificações de título, prioridade, data
|   `-- armazenamento.py          # Leitura e gravação dos dados em arquivo
|-- data/
|   `-- tarefas.json              # Arquivo onde as tarefas ficam salvas
`-- tests/
    `-- test_tarefas.py           # Testes das funções principais
```

Justificativa da estrutura: separar src/ (código), docs/ (documentação), data/ (dados) e tests/ (testes) mantém o repositório legível para qualquer pessoa que o abra pela primeira vez. O `README.md` na raiz é o cartão de visitas do projeto no GitHub, e o `.gitignore` evita que arquivos gerados automaticamente pelo Python poluem o histórico de versões.

4.2 Plano de Aprendizado para Git e GitHub

Na Fase 2, espero aprender de fato como utilizar e versionar usando o git. Pretendo dominar o fluxo básico (`git init`, `git add`, `git commit`, `git push`, `git pull`), aprender a criar *branches* para desenvolver novas funcionalidades sem quebrar a versão que já funciona, e enfrentar de forma controlada o meu primeiro conflito de código, entendendo por que ele acontece e como resolvê-lo com calma em vez de recomeçar do zero. Também quero publicar o TaskTracker no GitHub com um `README.md` bem escrito e um `.gitignore` correto, para que o repositório sirva tanto como base técnica das próximas fases quanto como peça de portfólio profissional. Meu objetivo final é que, ao terminar a Fase 2, o versionamento seja um hábito natural do meu trabalho, algo que me dá

segurança para experimentar, porque sei que consigo voltar atrás, e não uma etapa burocrática executada no fim do projeto.

5. Link da Apresentação em Vídeo

Apresentação do planejamento gravada e publicada no YouTube, e também o link de acesso do Google Docs:

Link do vídeo no YouTube e do Google Docs:

`https://youtu.be/-BE4o7EPG0g`
`Planejamento Lógico TaskTracker`